Module 2: How Claude Code Works

Claude Code for Academic Researchers

From Interactive to File-Based Workflows

What you do now (Stata / R):

- Open a session, load data into memory
- Run commands interactively, see results immediately

How Claude Code works:

- Reads and writes **files on disk**
- No persistent session with your data loaded
- Reads your files $\rightarrow$ writes/modifies scripts $\rightarrow$ optionally runs them

Practical Implication

Think in Projects, Not Sessions

You need to think in terms of:

- Project directories
- File paths
- Scripts that run end-to-end

Not interactive exploration.

The Context Window

Claude Code can only "see" a limited amount of text at once.

The context window includes:

- Your instructions
- The files it reads
- Its own responses

When a conversation gets long or a project gets large, it loses earlier context.

Why the Context Window Matters

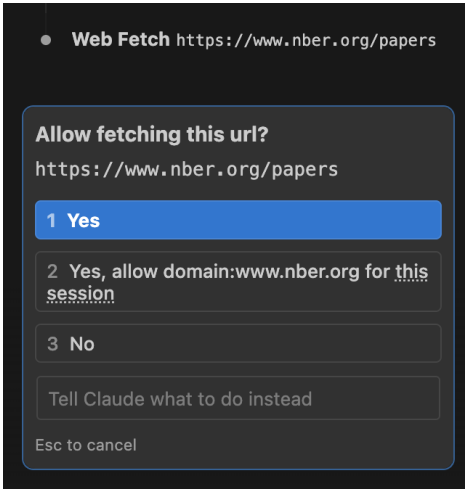
The Consistency Problem

In a complex project with many files, Claude Code may change one file in a way that is **inconsistent** with another file it can no longer see.

Errors compound when the agent loses track of earlier decisions.

We will cover strategies for managing this in Module 3.

When Claude Code Asks for Permission



Before taking actions (running code, writing files), Claude Code asks you.

Your three options:

1. **Yes**: allow this one time
2. **Yes, always allow**: allow this type of action for the rest of the session (no more prompts)
3. **No**: reject, and tell Claude Code what to do differently

"No" is not a dead end.

Use it to redirect: *"No, use a left join instead"* or *"No, don't overwrite that file, create a new one."*

Common Actions and Risk Levels

Action	Example you'll see	What it does	Risk level
`Read`	`Read scripts/clean.py`	Looks at a file	Safe
`Glob`	`Glob data/**/*.csv`	Finds files by name	Safe
`Grep`	`Grep "county_fips"`	Searches inside files	Safe
`Edit`	`Edit clean.py line 42`	Changes existing code	Review first
`Write`	`Write scripts/merge.py`	Creates a new file	Review first
`Bash`	`python scripts/clean.py`	Runs a command	Read before approving
`Bash`	`pip install pandas`	Installs a package	Read before approving
`Bash`	`rm data/old_panel.csv`	Deletes a file	Read before approving
`Bash`	`mv output.csv data/`	Moves a file	Read before approving
`Bash`	`git push`	Uploads to remote server	Read before approving

Safe to "always allow":

Reading and searching. Claude Code is just looking, not changing anything.

Always read before approving:

Bash commands can do *anything*: run code, delete files, install software. Check what it is actually running.

Protecting Sensitive Files

"Always allow reads" assumes there is nothing sensitive to read.
If your project has restricted data or IRB-protected files, you need to hide them.

How: Create a plain text file called `.claudeignore` in your project folder.
Each line is a pattern. Any file matching a pattern becomes **invisible** to Claude Code. It cannot read, search, or reference it.

Example `.claudeignore`:

```
# Everything in the restricted folder
data/restricted/

# Any file whose name starts with "identifiable_"
data/raw/identifiable_*

# A specific file
data/raw/ssn_crosswalk.csv
```

Setting Up .claudeignore

1. Open your project folder in VS Code
2. Create a new file named exactly `.claudeignore`
   (note the dot at the start, which makes it a hidden file)
3. Add one pattern per line:
   - A folder name ending in / hides everything inside it
   - A * matches any characters (e.g., `*.pem` hides all `.pem` files)
   - Lines starting with # are comments
4. Save the file. Claude Code reads it automatically

Do this **before** your first Claude Code session.

If Claude Code has already read a file in the current conversation, adding it to `.claudeignore` later does not undo that.

The Three Execution Modes

The most important configuration decision you will make.

Plan Mode: Reviews approach with you *before* acting

Default Mode: Acts, but asks permission for "significant" steps

Auto-Accept: Executes everything without asking

↓ More autonomy, less oversight ↓

How it works:

- Claude Code analyzes your request
- Proposes a detailed plan (files to read, changes to make, approach)
- Waits for your explicit approval before acting
- You review, approve, modify, or reject

When to use:
Any task where the *approach* matters, not just the output.
Data transformation, variable construction, merges, anything analytical.

Why It Matters

The plan is your checkpoint. This is where you catch "I am going to do an inner join" *before* it happens, not after 300 observations are gone.

How it works:

- Claude Code executes steps autonomously
- Pauses for actions *it* considers significant
- Uses its own judgment about when to ask

When to use:
Routine tasks in established projects where you trust the general approach.

The Risk for Academics

Claude Code's judgment about what is "significant" may not match yours.
It might pause before writing a file but **not** before silently recoding a variable.

How it works:

- Executes everything without asking
- Reads, writes, runs, and modifies freely
- Stops only on errors

When to use:

- File reformatting
- Boilerplate generation
- Documentation

When not to use:
Anything involving your data or analysis.
The speed gain is not worth the loss of checkpoints.

The Key Insight on Modes

Different Relationships with the Agent

Plan mode = you **direct** the work
Auto-accept = you **review** the work after the fact

For published research, directing is almost always safer than reviewing after the fact.

Why: Errors you *prevent* at the planning stage are cheaper than errors you *catch* in the output, or worse, errors you don't catch at all.

Live Demo: Plan Mode vs. Auto-Accept

Same task, two modes.

Watch for:

- What does a plan look like?
- How do you evaluate it?
- What does the experience feel like in each mode?

Choosing a Model

Claude Code lets you switch between three model tiers.
Each trades off **quality**, **speed**, **cost**, and **context window size**.

	Haiku 4.5	**Sonnet 4.6**	**Opus 4.6**
Context window	200k tokens	200k tokens	200k–1M tokens
Speed	Fastest	Fast	Slowest
Cost	Lowest	Moderate	Highest
Reasoning	Basic	Strong	Strongest

To switch models in Claude Code:
Type `/model` and select, or press `Shift+Tab` to toggle.

When to Use Each Model

Haiku

Fast, cheap, good enough for simple tasks.

- Reformatting files
- Renaming variables
- Generating boilerplate
- Quick questions about syntax

Sonnet

The everyday workhorse for most research tasks.

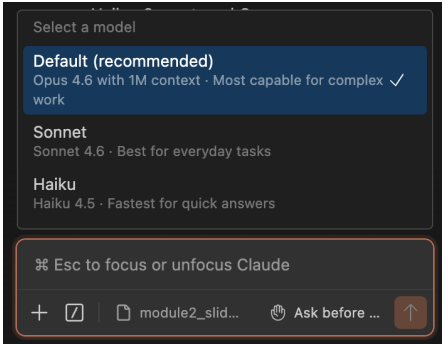
- Writing data cleaning scripts
- Building panels and merges
- Debugging code
- Drafting documentation

Opus

Maximum reasoning power; use when it matters.

- Complex multi-file refactors
- Subtle specification decisions
- Reviewing analytical code
- Large-context projects (up to 1M tokens)

Model Choice: The Research Perspective



The Key Tradeoff

Smarter models are slower and more expensive.
A wrong answer fast is worse than a right answer slow.

Rules of thumb:

- **Default to Sonnet**: best balance of quality and speed
- **Upgrade to Opus** for careful reasoning or many files
- **Drop to Haiku** for mechanical, easy-to-verify tasks

Cost Management

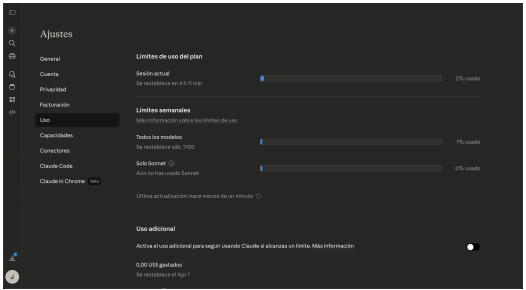
A Real Constraint

On the $20/month plan, you can burn through your allocation in 30–45 minutes of active use if you are not strategic.

Principles:

1. Not everything needs Claude Code. Small edits are faster by hand
2. A well-written CLAUDE.md reduces back-and-forth (= fewer tokens)
3. If Claude Code is spinning on a wrong approach, **intervene early**
4. Batch related tasks into one well-scoped request
5. Plan mode adds tokens but prevents costly wrong approaches

Usage Limits and Model Cost



How limits work:

- Token allowance resets on a **5-hour rolling window**
- A separate **weekly cap** limits total usage
- When you hit either limit, Claude Code pauses until the window resets

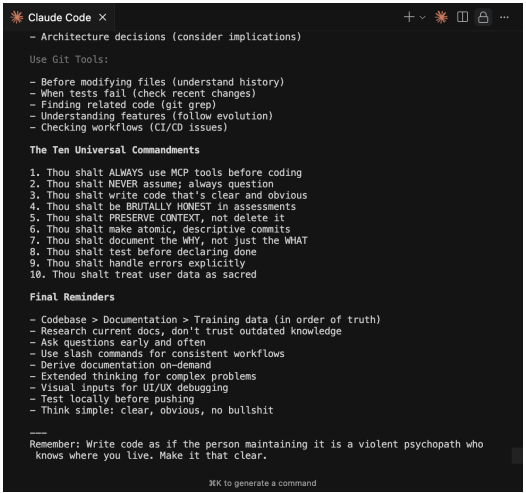
Which models burn tokens fastest:

- **Opus** uses $\sim 5\times$ more tokens per task than Haiku
- **Sonnet** is the middle ground
- **Haiku** stretches your budget the furthest

Match the model to the task.

Don't use Opus to rename variables.

CLAUDE.md: Your Lab Notebook for the Agent



Credit: u/rishabhsonker

A plain text file in your project directory that Claude Code reads **automatically at the start of every conversation**.

It encodes your conventions, definitions, and constraints so you do not have to repeat them each time.

The single most effective way to prevent errors before they happen.

Note: This does not guarantee Claude Code will always follow these instructions, but it significantly increases the chances.

What Belongs in CLAUDE.md

Include:

- Project description & research question
- Data structure: files, key variables, units, expected ranges
- Coding conventions
- Constraints: "never drop observations without logging"
- "All merges must report row counts"
- What Claude Code should **not** do

How it interacts with the context window

CLAUDE.md is loaded into the context window before your first message.
It is always "visible," but it takes up space, so keep it focused and concise.

CLAUDE.md Example

```
# CLAUDE.md - County Migration Project

## Research Question
Effect of historical migration on county-level outcomes.

## Data
- data/raw/census_panel.csv: county-year panel, 1920-2020
- Key vars: fips (5-digit), year, population, migration_share
- All monetary values in 2020 USD

## Constraints
- Never drop observations without logging count and reason
- All merges must report pre- and post-merge row counts
- Always use robust standard errors
- Do not choose model specifications
- Do not interpret coefficients
```

Exercise: Build the County Panel From Scratch

In Module 1 you ran a pre-built script. Now **you** build it.

Same data, fresh start. Work in pairs using plan mode.

1. Write a `CLAUDE.md` for the project (use the template)
2. Switch Claude Code to **plan mode**
3. Ask Claude Code to build a county-year panel that merges election returns with IPUMS census demographics
4. **Before approving the plan**, you will need to:
   - Read the codebooks to understand variable codings
     (What does `EDUC` $\geq$ 10 mean? What does `RACE == 1` code?)
   - Decide how to define key variables
     (Two-party vote share? College = bachelor's or higher?)
   - Specify the merge: which keys, which join type, expected row counts
5. Review the plan, reject or revise as needed, then approve
6. Run the script and verify the output

Evaluating the Plan

When Claude Code proposes its plan, ask yourself:

- Is it reading the **codebooks** or just guessing at variable meanings?
- How is it coding `EDUC`, `RACE`, `HISPAN`?
  Do those match *your* definitions?
- What join type is it using? Inner, left, or outer?
  What happens to counties that appear in one dataset but not the other?
- Is it filtering `mode == "TOTAL"` for election data?
- Does it handle `COUNTYFIP == 0` (unidentified counties)?

This is where your domain knowledge matters.
Claude Code can write the code, but you decide what the variables mean.

Debrief

- What variable coding decisions did you make? Did Claude Code's default match yours?
- Did anyone reject or modify the plan? What did you change?
- How did your panel compare to the Module 1 version?

The Lesson

Writing a CLAUDE.md forces you to be explicit about conventions you have kept implicit.

Reviewing plans forces you to think about approach *before* you see output.

Domain knowledge is key for preventing errors.

Module 2 Summary

1. Claude Code works with **files**, not interactive sessions
2. The **context window** limits what it can hold, so keep tasks focused
3. **Plan mode** is the default for research work
4. **CLAUDE.md** is your most powerful tool for preventing errors
5. **Cost management** is a core skill, not an afterthought

Next: Module 3, Automating Research Tasks